ANALISIS ALGORITMA BACKPROPAGATION PADA STUDI KASUS BANKNOTE AUTHENTICATION

Analysis of the Backpropagation Algorithm for Banknote Authentication Classification

Ahmad Ali Husain | D082251018
Program Studi Teknik Informatika, Universitas Hasanuddin, Gowa
Mata Kuliah Algoritma Komputasi

***Abstrak*—**Backpropagation adalah algoritma untuk menghitung gradien fungsi loss terhadap seluruh bobot jaringan saraf tiruan secara efisien dengan menerapkan aturan rantai dari lapisan output ke lapisan input. Laporan ini menganalisis kebenaran, kompleksitas waktu, dan kompleksitas ruang Backpropagation pada jaringan satu hidden layer, lalu mengujinya pada dataset UCI Banknote Authentication yang berisi 1.372 sampel dengan empat fitur numerik hasil ekstraksi citra (variance, skewness, curtosis, entropy) dan label kelas 0/1. Model diimplementasikan dengan NumPy menggunakan aktivasi sigmoid, loss Binary Cross Entropy (BCE), dan full-batch Gradient Descent. Data dibagi secara stratified menjadi 823 data latih, 274 validasi, dan 275 uji. Pemeriksaan gradien dengan finite differences menghasilkan selisih maksimum $9{,}08 \times 10^{-12}$, sehingga turunan analitik konsisten dengan turunan numerik. Dari 45 percobaan validasi ($H \in \{4, 8, 16\}$, learning rate $\in \{0{,}1; 0{,}5; 1{,}0\}$, lima seed) dipilih arsitektur 4–16–1 dengan learning rate 1,0. Setelah 350 epoch, model mencapai akurasi uji 98,91% (272/275) dengan recall kelas 1 sebesar 100%, sedikit di atas Logistic Regression sebagai pembanding (98,55%; 271/275). Selisih ini hanya satu sampel sehingga belum menunjukkan keunggulan yang bermakna. Secara teoretis biaya per epoch adalah $\Theta(NdH)$ dan ruang tambahan $\Theta(NH + dH)$. Benchmark lima ukuran × lima ulangan menunjukkan bahwa kenaikan N sekitar 5,02 kali hanya menaikkan waktu per epoch sekitar 1,32 kali, karena pada ukuran sekecil ini waktu didominasi biaya tetap pemanggilan NumPy. Keterbatasan studi meliputi satu dataset tabular, satu arsitektur, dan pengukuran waktu pada satu lingkungan Google Colab.

Kata Kunci—Backpropagation, jaringan saraf tiruan, kompleksitas algoritma, Banknote Authentication, klasifikasi.

Abstract*—Backpropagation efficiently computes loss gradients with respect to the weights of an artificial neural network through reverse application of the chain rule. This report examines the correctness, time complexity, and space complexity of Backpropagation in a network with one hidden layer and evaluates its implementation using the UCI Banknote Authentication dataset, which contains 1,372 samples with four numerical image-derived features and a binary class label. The NumPy-based model uses sigmoid activation, Binary Cross Entropy, and full-batch Gradient Descent. Data were split into 823 training, 274 validation, and 275 test samples. A finite-difference gradient check yielded a maximum absolute discrepancy of 9.08×10^{-12}. Across 45 validation runs, a 4–16–1 architecture with a learning rate of 1.0 was selected. After 350 epochs, the model attained 98.91% test accuracy (272/275), compared with 98.55% (271/275) for a Logistic Regression baseline. The difference of one prediction is too small to establish a general advantage. The theoretical per-epoch cost is $\Theta(NdH)$, while practical timing on five sample sizes displayed substantial fixed overhead. The study is limited to one tabular dataset and one Google Colab CPU environment.*

Keywords—Backpropagation, neural network, algorithm complexity, banknote authentication, binary classification.

1. PENDAHULUAN

1.1 Latar Belakang

Jaringan saraf tiruan belajar dengan menyesuaikan bobot dan bias agar nilai fungsi loss pada data latih mengecil. Metode optimasi berbasis gradien, seperti Gradient Descent, memerlukan turunan parsial loss terhadap setiap parameter pada setiap langkah. Cara paling naif untuk memperolehnya adalah finite differences: setiap parameter digeser sedikit lalu loss dihitung ulang. Untuk jaringan dengan P parameter, cara ini membutuhkan sekitar 2P kali forward pass per langkah, sehingga biayanya tumbuh sebanding dengan jumlah parameter dikali biaya satu forward pass.

Backpropagation [2] menghindari pemborosan tersebut. Dengan menyimpan hasil antara dari forward pass dan menerapkan aturan rantai secara terbalik (dari output ke input), seluruh gradien dapat diperoleh dengan biaya yang hanya sekelipatan konstan dari satu forward pass. Karena itu Backpropagation layak

dianalisis sebagai algoritma: kebenarannya dapat dibuktikan dari aturan rantai, biaya waktunya dapat dinyatakan sebagai fungsi jumlah sampel N, jumlah fitur d, jumlah neuron tersembunyi H, dan jumlah epoch E, dan biaya ruangnya ditentukan oleh aktivasi yang harus disimpan untuk tahap backward. Relevansi isu biaya ini masih terlihat pada penelitian terbaru yang berupaya menekan kebutuhan memori dan waktu Backpropagation [3]–[7].

Dataset UCI Banknote Authentication [1] dipilih sebagai studi kasus karena sifatnya cocok untuk menguji algoritma, bukan untuk membangun sistem pendeteksi uang palsu. Dataset ini tidak berisi foto mentah, melainkan empat fitur numerik kontinu (variance, skewness, curtosis, entropy) yang sudah diekstraksi dari citra sebelum masuk ke model, dengan target dua kelas (kelas 0 dan kelas 1). Target biner sesuai dengan output sigmoid dan loss Binary Cross Entropy. Ukurannya yang kecil (1.372 × 4) membuat full-batch Gradient Descent dapat dijalankan secara utuh, gradien dapat diperiksa secara numerik, dan eksperimen dapat diulang berkali-kali (berbagai ukuran N, nilai H, learning rate, dan seed) dalam waktu singkat. Di sisi lain, dataset ini relatif mudah dipisahkan sehingga model linear pun berkinerja tinggi. Hal ini justru berguna sebagai pengingat bahwa akurasi tinggi tidak otomatis membuktikan perlunya hidden layer, sehingga perbandingan dengan Logistic Regression menjadi bagian penting dari analisis.

1.2 Definisi Masalah

Input: empat variabel numerik variance, skewness, curtosis, entropy. Target: class $\in \{0,1\}$. Model satu hidden layer dengan 1 output sigmoid. Tujuan komputasi: menghitung $\partial L/\partial W$ dan $\partial L/\partial b$ secara benar, lalu mengukur biaya dan kinerja algoritma.

1.3 Tujuan dan Batasan

Penelitian ini bertujuan untuk: (1) menurunkan dan mengimplementasikan Backpropagation untuk jaringan d–H–1 dengan aktivasi sigmoid dan loss BCE, lalu memverifikasi gradiennya dengan finite differences; (2) menganalisis kompleksitas waktu dan ruang per epoch dan untuk E epoch; (3) mengukur waktu per epoch pada lima ukuran data dan membandingkannya dengan prediksi teoretis; serta (4) mengukur kinerja klasifikasi dan sensitivitas terhadap jumlah neuron tersembunyi H dan learning rate, dengan Logistic Regression sebagai pembanding.

(a) Data yang dipakai hanya UCI Banknote Authentication (1.372 sampel, 4 fitur jadi); tidak ada pemrosesan citra. (b) Data dibagi secara stratified menjadi 823 latih, 274 validasi, dan 275 uji (sekitar 60/20/20). Standardisasi z-score memakai rata-rata dan simpangan baku dari data pelatihan, dan data uji tidak pernah dipakai untuk memilih parameter. (c) Arsitektur dibatasi pada satu hidden layer sigmoid dengan $H \in \{4, 8, 16\}$ dan satu neuron output sigmoid. (d) Optimasi memakai full-batch Gradient Descent dengan learning rate tetap (0,1; 0,5; atau 1,0); tidak ada mini-batch. (e) Jumlah epoch tetap: 350 epoch untuk model akhir dan 120 epoch untuk benchmark waktu, sehingga analisis biaya memperlakukan E sebagai parameter dan bukan sebagai jumlah iterasi sampai konvergen. (f) Waktu diukur pada satu lingkungan Google Colab berbasis CPU; jenis prosesornya tidak tercatat. (g) Skema ini adalah eksperimen akademik untuk menganalisis algoritma, bukan sistem verifikasi uang kertas komersial. Model tidak menangani akuisisi gambar, tidak diuji pada mata uang atau perangkat pemindai lain, dan tidak dirancang menghadapi upaya pemalsuan yang disengaja.

2. LANDASAN TEORI

2.1 Jaringan Saraf Tiruan

2.2 Forward Propagation

Dalam notasi matriks: a1 = sigmoid(XW1 + b1), p = sigmoid(a1W2 + b2). Jelaskan ukuran setiap matriks dengan N baris data dan H neuron hidden.

2.3 Loss Binary Cross Entropy

BCE digunakan karena target adalah dua kelas dan output sigmoid. Kode menghitung logaddexp(0, z2) − y × z2 secara numerik stabil, kemudian dirata-ratakan terhadap sampel.

2.4 Backpropagation, Chain Rule, dan Gradient Descent

Backward menghitung gradien, sementara step menggunakan learning rate untuk memperbarui parameter. Jangan menyatakan bahwa Backpropagation secara sendirian selalu menjamin konvergensi ke minimum global.

2.5 Penelitian Terdahulu

Rumelhart, Hinton, dan Williams [2] memperkenalkan prosedur back-propagating errors untuk melatih jaringan berlapis yang memiliki unit tersembunyi. Inti kontribusinya adalah bahwa error di output dapat dirambatkan mundur melalui aturan rantai sehingga setiap bobot memperoleh sinyal koreksi, dan unit tersembunyi dapat mempelajari representasi internal sendiri. Implementasi pada laporan ini adalah bentuk paling sederhana dari prosedur tersebut: satu hidden layer dengan gradien eksak.

Gusak dkk. [4] menyurvei teknik untuk membuat pelatihan jaringan besar lebih efisien. Mereka menekankan bahwa memori pelatihan tidak hanya dipakai untuk bobot, tetapi juga untuk aktivasi dan tensor antara yang harus disimpan untuk tahap backward, sehingga banyak model tidak muat pada satu GPU atau hanya dapat dilatih dengan batch kecil. Survei ini mengelompokkan dan membandingkan berbagai strategi penghematan memori serta pemanfaatan komputasi dan komunikasi pada satu maupun banyak GPU.

Beberapa penelitian lain mengurangi biaya Backpropagation dengan sengaja melepaskan sebagian kepresisian gradien. Cheng dkk. [3] mengusulkan Stochastic Backpropagation untuk model video: semua jalur forward dipertahankan, tetapi jalur backward pada tiap layer dibuang secara acak sehingga aktivasinya tidak perlu disimpan. Pendekatan ini memanfaatkan redundansi antar-frame video dan dilaporkan menghemat memori GPU hingga 80% dengan penurunan akurasi kurang dari 1%. Meng dkk. [6] mengusulkan Spatial Learning Through Time untuk Spiking Neural Network. Mereka mengamati bahwa jalur gradien melalui dimensi waktu pada BPTT hanya berkontribusi kecil, sehingga jalur itu dapat diabaikan dan memori menjadi tidak bergantung pada jumlah time step; pada ImageNet memori dan waktu pelatihan turun lebih dari 70% dan 50% dibandingkan BPTT. Yang dkk. [7] menangani fine-tuning model pralatih besar dengan memisahkan forward dan backward: fungsi aktivasi GELU/SiLU tetap dipakai pada forward, tetapi turunannya pada backward didekati agar aktivasi yang disimpan lebih hemat, ditambah berbagi memori aktivasi antar-layer yang berdekatan. Mereka melaporkan penurunan puncak memori hingga sekitar 30%.

Fournier dkk. [5] meninjau alternatif yang sama sekali tidak memakai backward pass, yaitu forward gradient yang menaksir gradien dari turunan berarah dengan mode forward. Masalah utamanya adalah memilih arah tebakan yang baik di ruang berdimensi tinggi. Mereka menunjukkan bahwa arah tebakan dari gradien loss lokal (jaringan bantu kecil) jauh lebih baik daripada noise acak, tetapi kualitasnya tetap bergantung pada tebakan tersebut. Dengan demikian, gradien eksak dari Backpropagation tetap menjadi acuan.

Semua penelitian [3]–[7] berangkat dari masalah jaringan besar, yaitu jutaan sampai miliaran parameter dan aktivasi yang memenuhi memori GPU. Kondisi itu tidak terjadi pada MLP 4–16–1 dalam laporan ini, yang hanya memiliki 97 parameter dan matriks aktivasi tersembunyi 1.097 × 16 (sekitar 140 KB dalam float64). Teknik yang membuang atau mendekati gradien akan menambah galat tanpa manfaat yang berarti, dan juga akan menggagalkan pemeriksaan gradien yang dipakai sebagai bukti kebenaran pada Bab III. Oleh karena itu, temuan kuantitatif penelitian tersebut tidak dapat dialihkan langsung ke kasus Banknote. Yang dapat dipakai adalah kerangka berpikirnya: memori Backpropagation didominasi aktivasi yang disimpan untuk tahap backward, sehingga ada pertukaran antara memori, komputasi, dan ketepatan gradien. Kerangka ini dipakai pada analisis ruang di Subbab 3.6.

3. ANALISIS ALGORITMA

3.1 Notasi, Dimensi, dan Asumsi

Simbol	Makna
N	Jumlah data dalam satu batch penuh
d	Jumlah fitur input (=4 dalam Banknote)
H	Jumlah neuron tersembunyi (=4,8,16 dalam uji sensitivitas)
E	Jumlah epoch
W1 / W2	Bobot input→hidden / hidden→output
L	Rata-rata Binary Cross Entropy

3.2 Contoh Hitung Manual

Agar selaras dengan eksperimen Banknote, contoh hitung manual ini memakai BCE dan bukan MSE/2.

Jaringan yang dipakai berukuran 2–2–1 dengan satu sampel (N = 1). Semua angka di bawah adalah ilustrasi buatan, bukan data Banknote, dan dibulatkan sampai enam desimal.

Data dan parameter awal.

x = [1,0 2,0], y = 1; W1 = [[0,1 0,3], [0,2 0,4]] (baris = input, kolom = neuron tersembunyi), b1 = [0,1 0,1], W2 = $[0,5\ \ -0,4]^T$, b2 = 0.

Forward.

$$z1 = xW1 + b1 = [1\cdot0,1 + 2\cdot0,2 + 0,1 \quad 1\cdot0,3 + 2\cdot0,4 + 0,1] = [0,6 \quad 1,2]$$

$$a1 = \sigma(z1) = [0,645656 \quad 0,768525]$$

$$z2 = a1W2 + b2 = 0,5\cdot0,645656 - 0,4\cdot0,768525 = 0,015418; \quad p = \sigma(z2) = 0,503854$$

$$L = \log(1 + e\text{^}z2) - y\cdot z2 = -\ln p = 0,685468$$

Backward.

Karena BCE dipasangkan dengan sigmoid, turunan loss terhadap z2 menjadi sederhana (lihat Subbab 3.4):

$$\delta2 = (p - y)/N = 0,503854 - 1 = -0,496146$$

$$\partial L/\partial W2 = a1^T\delta2 = [-0,320339 \quad -0,381300]^T; \quad \partial L/\partial b2 = -0,496146$$

$$\delta2W2^T = [-0,248073 \quad 0,198458]; \quad a1 \odot (1 - a1) = [0,228784 \quad 0,177894]$$

$$\delta1 = (\delta2W2^T) \odot a1 \odot (1 - a1) = [-0,056755 \quad 0,035305]$$

$$\partial L/\partial W1 = x^T\delta1 = [[-0,056755 \quad 0,035305], [-0,113510 \quad 0,070609]]; \quad \partial L/\partial b1 = [-0,056755 \quad 0,035305]$$

Pemeriksaan finite differences

(beda tengah, $\varepsilon = 10^{-5}$): $\partial L/\partial W1[1,1] \approx -0,056755$ dan $\partial L/\partial W2[1] \approx -0,320339$. Keduanya sama dengan hasil analitik sampai enam desimal.

Satu langkah update

($\eta = 1,0$): W2 menjadi $[0,820339\ \ -0,018700]^T$, b2 = 0,496146, W1 = [[0,156755 0,264695], [0,313510 0,329391]], dan b1 = [0,156755 0,064695]. Setelah update, p naik dari 0,503854 menjadi 0,745074 dan loss turun dari 0,685468 menjadi 0,294271.

Interpretasi.

δ2 bernilai negatif karena prediksi (≈0,50) masih di bawah target 1, sehingga semua parameter yang menaikkan z2 diperbesar. Neuron tersembunyi kedua terhubung ke output dengan bobot negatif (−0,4), sehingga δ1-nya positif dan bobot menuju neuron itu justru dikurangi agar aktivasinya turun. Gradien W1 pada baris input kedua bernilai dua kali baris pertama karena x2 = 2·x1. Hal ini menunjukkan bahwa gradien bobot sebanding dengan nilai input yang melewatinya.

3.3 Pseudocode

Per epoch: (1) forward; (2) loss; (3) turunkan delta output; (4) hitung gradien W2, b2; (5) turunkan delta hidden; (6) hitung gradien W1,b1; (7) update parameter menggunakan Gradient Descent.

3.4 Argumen Kebenaran

Model.

Untuk batch berisi N sampel: Z1 = XW1 + 1b1, A1 = σ(Z1), z2 = A1W2 + b2, p = σ(z2), dan L = (1/N) $\Sigma_i\ \ell_i$ dengan $\ell_i = \log(1 + e\text{^}z2_i) - y_i\cdot z2_i$. Bentuk ini identik dengan BCE biasa $-[y \ln p + (1 - y) \ln(1 - p)]$, karena $-\ln \sigma(z) = \log(1 + e\text{^}{-z})$ dan $-\ln(1 - \sigma(z)) = \log(1 + e\text{^}z)$. Kode menghitungnya dengan logaddexp agar stabil secara numerik ketika |z| besar.

Langkah 1 (output).

$\partial\ell_i/\partial z2_i = e\text{^}z/(1 + e\text{^}z) - y_i = \sigma(z2_i) - y_i = p_i - y_i$. Karena itu $\delta2 = \partial L/\partial z2 = (p - y)/N \in \mathbb{R}\text{^}(N\times1)$. Faktor σ′(z2) saling meniadakan, sehingga gradien output tidak melemah walaupun sigmoid sudah jenuh.

Langkah 2 (parameter output).

Karena $z2_i = \Sigma_h A1_{ih} W2_h + b2$, maka $\partial L/\partial W2_h = \Sigma_i \delta 2_i A1_{ih}$. Dalam bentuk matriks: $\partial L/\partial W2 = A1^T \delta 2$ (H×1) dan $\partial L/\partial b2 = \Sigma_i \delta 2_i$. Penjumlahan atas i muncul karena b2 dan W2 dipakai oleh setiap sampel, sehingga kontribusi semua jalur dijumlahkan.

Langkah 3 (hidden).

$\partial L/\partial A1_{ih} = \delta 2_i W2_h$, atau $\partial L/\partial A1 = \delta 2 W2^T$ (N×H). Karena $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ berlaku elemen per elemen, $\delta 1 = \partial L/\partial Z1 = (\delta 2 W2^T) \odot A1 \odot (1 - A1)$.

Langkah 4 (parameter hidden).

Karena $Z1_{ih} = \Sigma_j X_{ij} W1_{jh} + b1_h$, maka $\partial L/\partial W1 = X^T \delta 1$ (d×H) dan $\partial L/\partial b1 = 1^T \delta 1$ (jumlah kolom δ1).

Argumen kebenaran.

Perhitungan forward membentuk graf berarah tanpa siklus X → Z1 → A1 → z2 → L. Backward mengunjungi simpul-simpul tersebut dalam urutan topologis terbalik dan mempertahankan invarian berikut: setelah lapisan ℓ diproses, $\delta\ell$ sama persis dengan $\partial L/\partial Z\ell$. Invarian ini benar di output menurut Langkah 1. Jika invarian benar untuk lapisan $\ell + 1$, aturan rantai multivariabel (jumlah atas semua jalur, Langkah 3) membuatnya benar juga untuk lapisan ℓ. Gradien parameter tiap lapisan kemudian diperoleh dari δ lapisan itu dan input lapisannya (Langkah 2 dan 4). Dengan induksi mundur ini, keempat gradien yang dihasilkan sama dengan turunan parsial eksak, dengan batas hanya galat pembulatan floating point.

Pemeriksaan numerik.

Pada eksperimen Colab, gradien analitik dibandingkan dengan beda tengah $[L(\theta + \varepsilon) - L(\theta - \varepsilon)]/(2\varepsilon)$ dan selisih mutlak maksimumnya $9{,}078 \times 10^{-12}$ (ringkasan_hasil.json). Galat beda tengah sendiri berorde $O(\varepsilon^2)$ ditambah galat pembulatan sekitar ε_mesin/ε, sehingga selisih berorde 10^{-11} sampai 10^{-12} adalah besaran yang diharapkan jika turunan analitik benar. Kesalahan rumus, misalnya lupa faktor 1/N atau σ′, akan menghasilkan selisih yang sebanding dengan besar gradien itu sendiri. Contoh manual pada Subbab 3.2 juga cocok sampai enam desimal.

Kebenaran gradien bukan bukti konvergensi.

Pemeriksaan di atas hanya menunjukkan bahwa backward menghitung ∇L dengan benar pada titik yang diuji. Pemeriksaan itu tidak menjamin bahwa Gradient Descent mencapai minimum global. Loss sebagai fungsi (W1, b1, W2, b2) tidak konveks; menukar urutan neuron tersembunyi saja sudah menghasilkan banyak titik dengan loss yang sama. Untuk fungsi tak konveks yang halus, Gradient Descent dengan learning rate yang cukup kecil paling jauh dijamin mendekati titik stasioner, dan jaminan itu pun tidak diperiksa untuk η = 1,0. Bukti konvergensi dalam laporan ini bersifat empiris: loss pelatihan akhir turun monoton pada seluruh 350 epoch (0,639830 → 0,035134). Sebaliknya, hasil learning rate 0,1 berbeda cukup jauh antar-seed (misalnya akurasi validasi 81,4% sampai 96,0% untuk H = 4). Ini menunjukkan bahwa hasil akhir bergantung pada inisialisasi dan learning rate, bukan hanya pada kebenaran gradien.

3.5 Kompleksitas Waktu

Analisis memakai model RAM: setiap operasi aritmetika skalar, termasuk exp dan log, dianggap berbiaya konstan. Perkalian matriks (a×b)(b×c) dihitung dengan cara standar sebanyak a·b·c perkalian dan penjumlahan. NumPy/BLAS memang memakai jumlah operasi yang sama untuk matriks tipis seperti ini dan hanya mempercepat konstantanya. Rincian biaya satu epoch ditunjukkan pada Tabel 3.1.

$$T_epoch(N, d, H) = 4NdH + c' \cdot NH + \Theta(N) + \Theta(dH) = \Theta(NdH),$$

karena NdH ≥ NH ≥ N dan NdH ≥ dH untuk N, d, H ≥ 1. Untuk E epoch, $T_total = \Theta(E \cdot N \cdot d \cdot H)$. Biaya tumbuh linear terhadap masing-masing variabel: menggandakan N, H, atau E kira-kira menggandakan waktu. Pada Banknote d = 4 tetap, sehingga $T_total = \Theta(E \cdot N \cdot H)$. Suku dominannya berasal dari dua perkalian matriks XW1 dan $X^T \delta 1$. Untuk model akhir (N = 1.097, d = 4, H = 16, E = 350), kedua suku itu saja berjumlah sekitar $4 \cdot 1.097 \cdot 4 \cdot 16 \approx 2{,}8 \times 10^5$ operasi per epoch, atau sekitar $9{,}8 \times 10^7$ operasi untuk seluruh pelatihan.

Perbandingan dengan gradien naif.

Model memiliki P = dH + 2H + 1 parameter (97 untuk 4–16–1). Finite differences memerlukan 2P forward pass yang masing-masing berbiaya Θ(NdH), sehingga totalnya $\Theta(P \cdot NdH) = \Theta(N \cdot d^2 \cdot H^2)$ per epoch. Backpropagation memperoleh semua gradien dalam Θ(NdH), kira-kira sekelipatan konstan dari satu

forward pass. Untuk 4–16–1, perbedaannya sekitar dua orde besaran. Sebagai pembanding, satu iterasi Gradient Descent Logistic Regression berbiaya Θ(Nd).

Tabel 3.1. Biaya per epoch (forward + backward + update) untuk satu batch penuh.

Operasi	Dimensi	Perkiraan operasi aritmetika
$Z1 = XW1 + b1$	$(N\times d)(d\times H)$	$2NdH + NH$
$A1 = \sigma(Z1)$	$N\times H$	$c\cdot NH$ (c konstan untuk exp, tambah, bagi)
$z2 = A1W2 + b2$	$(N\times H)(H\times 1)$	$2NH + N$
$p = \sigma(z2)$ dan loss BCE	$N\times 1$	$\Theta(N)$
$\delta 2 = (p - y)/N$	$N\times 1$	$2N$
$\partial L/\partial W2 = A1^T\delta 2$, $\partial L/\partial b2$	$(H\times N)(N\times 1)$	$2NH + N$
$\delta 2W2^T$	$(N\times 1)(1\times H)$	NH
$\delta 1 = (\delta 2W2^T) \odot A1 \odot (1 - A1)$	$N\times H$	$3NH$
$\partial L/\partial W1 = X^T\delta 1$, $\partial L/\partial b1$	$(d\times N)(N\times H)$	$2NdH + NH$
Update W1, b1, W2, b2	$dH + 2H + 1$	$2(dH + 2H + 1)$

Jumlah seluruh baris tabel memberikan

Best, worst, dan average case.

Algoritma ini bersifat data-oblivious: urutan dan jumlah operasinya tidak bergantung pada nilai X atau y karena tidak ada percabangan yang ditentukan data. Jadi untuk N, d, H, dan E yang tetap, best case = worst case = average case = $\Theta(E\cdot N\cdot d\cdot H)$. Mudah atau sulitnya label hanya memengaruhi kualitas hasil, yaitu seberapa rendah loss dan seberapa tinggi akurasi setelah E epoch, bukan jumlah operasi. Kedua hal ini tidak boleh disamakan.

Batasan fixed epoch.

Hasil $\Theta(E\cdot N\cdot d\cdot H)$ mengandaikan E ditentukan sebelumnya, seperti pada kode (350 dan 120 epoch). Jika pelatihan dihentikan berdasarkan kriteria konvergensi, misalnya loss < τ, jumlah epoch E* menjadi bergantung pada data, inisialisasi, dan learning rate. Untuk loss tak konveks tidak ada batas umum E* yang sederhana, sehingga yang dapat dinyatakan dengan pasti hanyalah biaya per epoch. Notasi Θ juga menyembunyikan konstanta dan suku orde rendah, yang ternyata dominan pada ukuran data sekecil Banknote (lihat Subbab 4.5).

3.6 Kompleksitas Ruang

Kebutuhan memori dipisahkan menjadi data masukan, memori kerja algoritma, dan artefak eksperimen (Tabel 3.2). Nilai dihitung dalam float64 (8 byte per nilai).

Tabel 3.2. Komponen kebutuhan memori satu langkah pelatihan full-batch.

Komponen	Ukuran	Orde	Contoh model akhir (N=1.097, H=16)
Data X dan label y	$N\times d + N$	$\Theta(Nd)$	4.388 + 1.097 nilai
Aktivasi A1 (disimpan untuk σ′)	$N\times H$	$\Theta(NH)$	17.552 nilai ≈ 140 KB
z2 / p dan δ2	$N\times 1$	$\Theta(N)$	2 × 1.097 nilai
Sementara backward: $\delta 2W2^T$, δ1	$N\times H$	$\Theta(NH)$	17.552 nilai (dapat menimpa)
Parameter W1, b1, W2, b2	$dH + 2H + 1$	$\Theta(dH)$	97 nilai ≈ 0,8 KB
Gradien parameter	$dH + 2H + 1$	$\Theta(dH)$	97 nilai
Log eksperimen (kurva loss, CSV, PNG)	E dan jumlah run	$\Theta(E)$	350 baris loss; 25 + 45 baris CSV

Data X (N×d) adalah masukan, sehingga tidak dihitung sebagai ruang tambahan. Ruang tambahan yang benar-benar dibutuhkan Backpropagation adalah aktivasi tersembunyi A1, yang harus disimpan dari forward karena turunan sigmoid dihitung sebagai $A1 \odot (1 - A1)$, ditambah matriks sementara δ1 berukuran sama, vektor N×1 untuk output dan δ2, serta parameter dan gradien. Totalnya:

$$S_{tambahan} = \Theta(NH + dH) = \Theta(H(N + d)); \quad S_{total} = \Theta(N(d + H) + dH).$$

Ruang ini tidak bergantung pada E, karena setiap epoch menimpa aktivasi dan gradien epoch

sebelumnya. Penyimpanan yang bertambah terhadap E hanyalah kurva loss (satu nilai per epoch). Kurva loss, berkas CSV, JSON, dan grafik PNG adalah keperluan pelaporan eksperimen dan bukan bagian dari kompleksitas algoritma. Pada model akhir, suku aktivasi (≈ 2 × 140 KB) jauh lebih besar daripada parameter (< 1 KB). Pola ini sama dengan temuan literatur bahwa aktivasi mendominasi memori Backpropagation [4], walaupun besarnya di sini tidak menjadi masalah praktis. Jika memori terbatas, mini-batch berukuran B akan menurunkan suku aktivasi menjadi Θ(BH), dengan imbalan lebih banyak langkah update per epoch.

3.7 Algoritma Pembanding

Logistic Regression dari pustaka scikit-learn digunakan sebagai pembanding klasifikasi dua kelas. Perbandingan akurasi kedua model perlu dibaca dengan mempertimbangkan perbedaan kapasitas model, sehingga tidak sepenuhnya mengisolasi pengaruh algoritma optimasi.

4. IMPLEMENTASI DAN EKSPERIMEN

4.1 Data dan Lingkungan Uji

Dataset UCI Banknote Authentication: 1.372 baris, 4 fitur numerik dan 1 label 0/1. Eksperimen Colab melaporkan Python 3.13.16, NumPy 2.1.3, scikit-learn 1.6.1 dan platform Linux 6.6.122+ x86_64. Rincian tipe CPU/GPU tidak tercatat; jangan mengada-ada.

4.2 Persiapan Data

Pemisahan data secara stratified menghasilkan 823 sampel train, 274 validasi, dan 275 test. Rata-rata dan simpangan baku standardisasi dihitung pada train saja. Setelah pengaturan dipilih melalui validasi, train+validasi (1.097 sampel) digabung untuk pelatihan final; test disimpan untuk penilaian akhir.

4.3 Implementasi Inti

Implementasi mendefinisikan fungsi sigmoid, forward, loss BCE, backward, dan step. NumPy digunakan untuk komputasi matriks; rumus turunan Backpropagation diimplementasikan dalam kode inti. scikit-learn digunakan untuk pemisahan data dan pembanding Logistic Regression.

4.4 Rancangan Eksperimen

Pengujian	Variabel dan pengulangan
Validasi kebenaran	3 contoh kecil dan gradient check pada 4 input
Klasifikasi akhir	Satu evaluasi test setelah parameter dipilih via validation
Benchmark ukuran	5 ukuran × 5 run; epoch benchmark tetap
Sensitivitas	H = 4/8/16 × LR = 0,1/0,5/1,0; 5 seed tiap kombinasi
Pembanding	Logistic Regression pada data train+validation yang sama

4.5 Hasil Eksperimen dan Pembahasan

Ukuran N	Mean waktu (ms/epoch)	Simpangan baku (ms/epoch)
164	0.151511	0.006492
329	0.134106	0.037975
493	0.151849	0.021470
658	0.169323	0.001113
823	0.199549	0.006798

4.5.1 Ringkasan Hasil Eksperimen

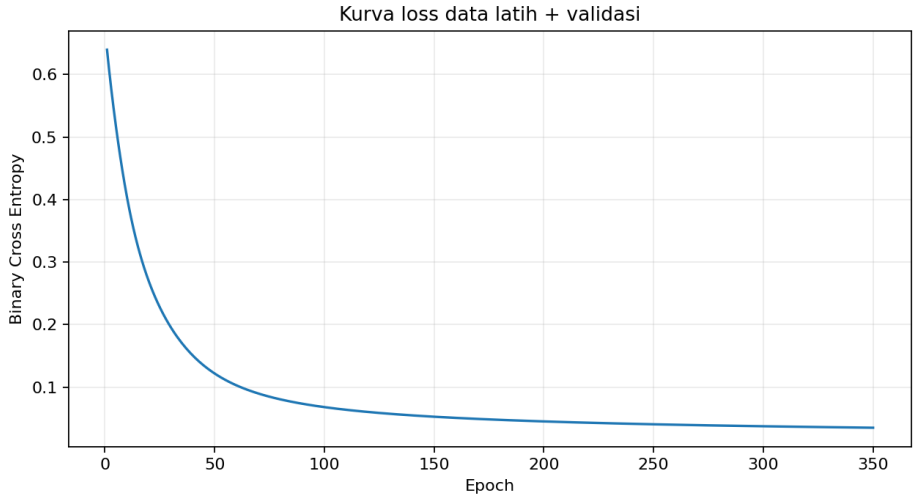
Data terverifikasi: dataset 1.372; train 823, validasi 274, test 275. Model terpilih setelah 45 percobaan validasi (3 pilihan H × 3 pilihan LR × 5 seed): H=16 dan LR=1,0. Akurasi test Backpropagation 272/275 (98,91%); baseline Logistic Regression 271/275 (98,55%). Confusion matrix Backpropagation: TP=122, TN=150, FP=3, FN=0. Akurasi sama sekali bukan bukti bahwa model dapat memproses foto uang mentah.

Kurva loss pelatihan akhir: epoch 1 = 0,639830; epoch 50 = 0,121920; epoch 350 = 0,035134. Besaran ini adalah Binary Cross Entropy rata-rata selama pelatihan akhir train+validasi.

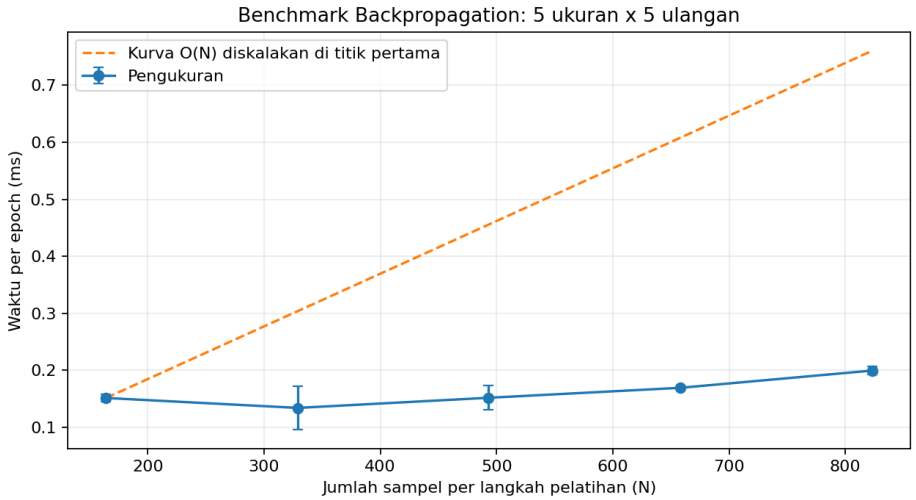
Tabel 4.3. Sensitivitas jumlah neuron dan learning rate: rata-rata dari lima seed.

Hidden (H)	LR	Akurasi validasi	Loss validasi
4	0.1	91.24%	0.31176
4	0.5	97.81%	0.05513
4	1.0	98.91%	0.03654
8	0.1	92.99%	0.25075
8	0.5	97.81%	0.05036
8	1.0	98.91%	0.03494
16	0.1	95.33%	0.18110
16	0.5	97.81%	0.04721
16	1.0	98.91%	0.03322

Gambar 4.1. Kurva Binary Cross Entropy pada pelatihan akhir (350 epoch); sumber: hasil Google Colab.



Gambar 4.2. Perbandingan waktu ukur dan garis referensi O(N); sumber: hasil Google Colab (25 pengukuran).



Metode	Akurasi Test	Precision kelas 1	Recall kelas 1
Backpropagation NumPy	98.91%	97.60%	100.00%
Logistic Regression	98.55%	96.83%	100.00%

Pembahasan waktu eksekusi.

Teori pada Subbab 3.5 memprediksi T_epoch = Θ(NdH). Dengan d = 4 dan H = 8 tetap pada benchmark, waktu per epoch seharusnya linear terhadap N. Jika seluruh waktu sebanding dengan N, kenaikan N dari 164 ke 823 (≈5,02 kali) seharusnya menaikkan waktu sekitar 5 kali. Hasil ukur hanya naik ≈1,32 kali (0,1515 → 0,1995 ms). Hal ini tidak bertentangan dengan teori, karena notasi Θ menggambarkan laju pertumbuhan untuk N besar dan menyembunyikan konstanta serta biaya tetap.

Model yang lebih realistis adalah T(N) = a + b·N. Regresi kuadrat terkecil pada 25 pengukuran di benchmark_raw.csv (dihitung penulis, bukan keluaran notebook) memberikan a ≈ 0,122 ms dan b ≈ 8,0 × 10^{-5} ms per sampel (R^2 ≈ 0,43). Pada N = 823, bagian yang sebanding dengan N hanya ≈0,066 ms, sedangkan biaya tetapnya ≈0,122 ms. Artinya, sekitar dua pertiga waktu per epoch belum berkaitan dengan jumlah data.

Biaya tetap itu masuk akal. Satu epoch memanggil belasan operasi NumPy (perkalian matriks, sigmoid, logaddexp, transpose, pengurangan, update). Setiap pemanggilan membawa overhead interpreter Python, pemeriksaan tipe dan dimensi, serta alokasi array baru dalam orde mikrodetik, berapa pun ukuran array-nya. Sebaliknya, kerja aritmetikanya sangat kecil: untuk N = 823 dan H = 8 sekitar 2 × 10^5 operasi, yang pada CPU modern hanya memerlukan puluhan mikrodetik. Orde ini sejalan dengan suku b·N ≈ 0,066 ms. Jadi rentang N pada benchmark masih berada di wilayah yang didominasi overhead, dan kemiringan linear baru akan dominan pada N yang jauh lebih besar.

Anomali N = 329 yang lebih cepat daripada N = 164 bukan semata-mata akibat pencilan. Simpangan bakunya besar (0,038 ms) karena ulangan pertama 0,202 ms, sedangkan empat ulangan lainnya hanya 0,114–0,123 ms, dan median 0,116 ms tetap lebih kecil daripada median N = 164 (0,149 ms). Penjelasan yang paling mungkin adalah efek pemanasan (warm-up). Ukuran N = 164 diukur pertama kali, ketika alokasi memori, cache, inisialisasi pustaka BLAS, dan frekuensi CPU belum stabil. Lonjakan ulangan pertama N = 329 dan ulangan kedua N = 493 (0,190 ms) juga menunjukkan gangguan sesaat pada mesin virtual Colab yang dipakai bersama. Mulai N = 329, median waktu naik secara monoton (0,116 → 0,141 → 0,169 → 0,197 ms), dan tren inilah yang sesuai dengan prediksi linear.

Kurva O(N) pada Gambar 4.2 diskalakan dengan menganggap seluruh waktu di titik pertama sebanding dengan N (a = 0). Karena itu kurva tersebut melebih-lebihkan pertumbuhan yang diharapkan dan tidak tepat dipakai untuk menilai kecocokan. Pembanding yang lebih adil adalah garis regresi a + b·N. Untuk memperkuat kesimpulan, benchmark sebaiknya: (1) menjalankan putaran pemanasan yang dibuang, (2) memakai lebih banyak ulangan dan melaporkan median, (3) memperluas N hingga puluhan ribu (misalnya dengan menduplikasi data untuk keperluan pengukuran waktu saja), dan (4) memvariasikan H untuk menguji linearitas terhadap H. Angka benchmark (H = 8, 120 epoch) juga tidak dapat dipakai langsung untuk menaksir waktu konfigurasi akhir (H = 16, 350 epoch).

Pembahasan kinerja klasifikasi dan sensitivitas.

Untuk learning rate 0,5 dan 1,0, akurasi validasi identik pada kelima seed dan ketiga nilai H, yaitu 268/274 (97,81%) dan 271/274 (98,91%). Akurasi sudah jenuh dan tidak lagi membedakan konfigurasi, sehingga perbedaan baru tampak pada loss validasi. Pada learning rate 1,0, loss validasi turun dari 0,03654 (H = 4) ke 0,03494 (H = 8) dan 0,03322 (H = 16), dan konfigurasi H = 16 dengan learning rate 1,0 yang terpilih memang memiliki loss validasi terendah. Dengan epoch tetap, learning rate jauh lebih berpengaruh daripada H. Pada learning rate 0,1, model belum konvergen (loss validasi 0,18–0,31) dan hasilnya sensitif terhadap seed. Pada kondisi itu, menambah H membantu (91,24% → 92,99% → 95,33%), kemungkinan karena lebih banyak neuron mempercepat penurunan loss dalam jumlah langkah yang sama.

Pada data uji, Backpropagation salah pada 3 dari 275 sampel dan Logistic Regression pada 4 sampel. Semua kesalahan berupa false positive (kelas 0 diprediksi sebagai kelas 1), sedangkan recall kelas 1 kedua metode 100%. Selisih satu sampel (0,36 poin persentase) terlalu kecil untuk menyatakan bahwa MLP lebih unggul, terlebih evaluasi uji hanya dilakukan sekali. Hasil Logistic Regression yang sangat tinggi menunjukkan bahwa kedua kelas pada fitur Banknote hampir terpisah secara linear, sehingga kapasitas nonlinear hidden layer hanya memberi tambahan kecil. Karena kapasitas kedua model berbeda, perbandingan ini juga tidak mengisolasi pengaruh algoritma Backpropagation itu sendiri.

Catatan interpretasi benchmark: kurva O(N) pada grafik hanya diskalakan pada titik pertama, bukan hasil regresi empiris. Benchmark menjaga H=8, epoch=120, sementara model akurasi final memakai H=16 dan epoch=350. Oleh sebab itu, jangan mengaitkan angka benchmark secara langsung dengan waktu konfigurasi model akhir.

5. KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan pengujian yang benar-benar dilakukan, dapat disimpulkan hal-hal berikut.

1. Gradien Backpropagation untuk jaringan d–H–1 dengan sigmoid dan BCE yang diturunkan dengan aturan rantai terbukti benar secara analitik dan cocok dengan finite differences (selisih maksimum $9{,}08 \times 10^{-12}$). Hasil ini membuktikan kebenaran perhitungan gradien, bukan konvergensi ke minimum global. Konvergensi hanya teramati secara empiris melalui loss yang turun monoton dari 0,6398 ke 0,0351 dalam 350 epoch.
2. Biaya waktu Backpropagation adalah $\Theta(NdH)$ per epoch dan $\Theta(E \cdot N \cdot d \cdot H)$ untuk E epoch tetap. Biaya ini sama untuk best, worst, dan average case karena algoritmanya data-oblivious. Ruang tambahannya $\Theta(NH + dH)$ dan didominasi aktivasi yang disimpan untuk tahap backward.
3. Pengukuran waktu pada N = 164–823 hanya sebagian mengonfirmasi teori. Mulai N = 329 waktu naik monoton, tetapi kenaikan N sekitar 5,02 kali hanya menaikkan waktu sekitar 1,32 kali karena biaya tetap (≈0,12 ms per epoch) masih dominan. Pertumbuhan linear asimtotik belum dapat dibuktikan penuh dengan rentang N sekecil ini.
4. Pada 275 data uji Banknote, model 4–16–1 mencapai akurasi 98,91%, precision kelas 1 sebesar 97,60%, dan recall kelas 1 sebesar 100%. Logistic Regression mencapai 98,55%. Selisih satu sampel belum cukup untuk menyatakan keunggulan. Angka ini hanya berlaku untuk fitur numerik pada dataset tersebut dan tidak dapat diartikan sebagai akurasi pendeteksian semua uang kertas atau kinerja sistem verifikasi yang nyata.
5. Dengan jumlah epoch tetap, learning rate lebih menentukan hasil daripada jumlah neuron tersembunyi. Learning rate 0,1 belum konvergen dan sensitif terhadap seed, sedangkan learning rate 0,5–1,0 memberikan akurasi validasi yang stabil.

5.2 Keterbatasan dan Pengembangan

Keterbatasan pengujian meliputi penggunaan satu dataset tabular dengan fitur yang sudah diekstraksi, satu hidden layer, pembanding dari keluarga model berbeda, serta eksperimen full-batch berbasis CPU. Hasil waktu komputasi juga dipengaruhi overhead pemanggilan operasi dan kondisi lingkungan pengujian.

DAFTAR PUSTAKA

[1] V. Lohweg, Banknote Authentication, UCI Machine Learning Repository, 2012. DOI:10.24432/C55P57.

[2] D. E. Rumelhart, G. E. Hinton, R. J. Williams, “Learning representations by back-propagating errors,” Nature, 323, 533–536, 1986. DOI:10.1038/323533a0.

[3] F. Cheng et al., “Stochastic Backpropagation: A Memory Efficient Strategy for Training Video Models,” CVPR, 2022, pp. 8301–8310.

[4] J. Gusak et al., “Survey on Efficient Training of Large Neural Networks,” IJCAI, 2022, pp. 5494–5501. DOI:10.24963/ijcai.2022/769.

[5] L. Fournier et al., “Can Forward Gradient Match Backpropagation?,” ICML, PMLR 202, 10249–10264, 2023.

[6] Q. Meng et al., “Towards Memory- and Time-Efficient Backpropagation for Training Spiking Neural Networks,” ICCV, 2023.

[7] Y. Yang et al., “Reducing Fine-Tuning Memory Overhead by Approximate and Memory-Sharing Backpropagation,” ICML, 2024.

[8] Dokumentasi pustaka NumPy, https://numpy.org/doc/stable/

LAMPIRAN

A. Tautan Repositori

Berikut link repositori github https://github.com/Quera14/backpropagation_Ali

B. Lembar Verifikasi Contoh Perhitungan

Lampiran ini merupakan langkah hitung contoh 2–2–1 menggunakan BCE. Contoh angka bersifat ilustratif dan bukan data Banknote. Pemeriksaan numerik sudah dijalankan, sedangkan verifikasi manual oleh mahasiswa perlu dilakukan secara mandiri.

B.1 Parameter Awal

Input x = [1; 2], target y = 1; W1 = [[0,1; 0,3], [0,2; 0,4]], b1 = [0,1; 0,1], W2 = [0,5; −0,4]^T, b2 = 0, learning rate η = 1,0.

C.1 Pemeriksaan Forward Propagation

Langkah	Operasi	Hasil (enam desimal)
Praktivasi hidden	xW1 + b1	[0,600000; 1,200000]
Aktivasi hidden	σ(z1)	[0,645656; 0,768525]
Praktivasi output	a1·W2 + b2	0,015418
Probabilitas output	σ(z2)	0.503854
Loss BCE	log(1+exp(z2)) − y·z2	0.685468

C.2 Pemeriksaan Backward dan Update

Delta output δ2 = (p − y)/N = -0.496146.

Gradien W2 = a1^{T}δ2 = [-0.320339; -0.381300]^T, gradien b2 = -0.496146.

Delta hidden δ1 = (δ2W2^T)⊙a1⊙(1−a1) = [-0.056755; 0.035305].

Gradien W1 = x^{T}δ1 = [[-0.056755; 0.035305],[-0.113510; 0.070609]].

Setelah satu update Gradient Descent, probabilitas = 0.745074 dan loss = 0.294271.

Pemeriksaan silang menggunakan finite differences (ε = 10^{-5}) tersedia pada berkas `verifikasi_manual/periksa_gradien_manual.py`. Berkas log hasil eksekusinya menyimpan selisih analitik-versus-numerik untuk W1, b1, W2, dan b2.

D. Jejak Eksperimen pada Google Colab

D.1 Asal Hasil dan Lingkungan

Data sumber: UCI Banknote Authentication (1.372 sampel); train/validasi/test = 823/274/275. Sistem: Linux-6.6.122+-x86_64-with-glibc2.39; Python 3.13.16, NumPy 2.1.3, scikit-learn 1.6.1.

Pengaturan pengujian: 3 pilihan H × 3 learning rate × 5 seed = 45 percobaan sensitivitas; pengukuran waktu 5 ukuran × 5 ulangan. Data mentah beserta plot disimpan tanpa modifikasi di folder `hasil_eksperimen` repositori pendamping.

D.2 Rekapitulasi Benchmark Terbaru

N	Mean (ms/epoch)	SD (ms/epoch)	Ulangan
164	0.151511	0.006492	5
329	0.134106	0.037975	5
493	0.151849	0.021470	5
658	0.169323	0.001113	5
823	0.199549	0.006798	5

Interpretasi benchmark harus membedakan kurva referensi O(N) yang diskalakan dari ukuran pertama dan pengukuran empiris. Pada ukuran data kecil, pemanggilan NumPy dan overhead waktu tetap dapat mendominasi.

D.3 Pemeriksaan Akurasi dan Provenans

Konfigurasi terpilih: 4-16-1, learning rate 1.0, 350 epoch. Akurasi uji Backpropagation 98.91% (272/275) dan Logistic Regression 98.55% (271/275). Selisih gradien maksimum 9.078e-12.

E. Panduan Pemeriksaan dan Reproduksi

E.1 Struktur Berkas Pendukung

`notebooks/`: notebook Google Colab yang digunakan sebagai alur eksperimen.

`src/`: implementasi Backpropagation NumPy.

`hasil_eksperimen/`: CSV, JSON, dan PNG keluaran eksperimen.

`verifikasi_manual/`: lembar hitung, skrip finite differences, dan output pemeriksaan.

`dokumentasi/`: protokol eksperimen dan interpretasi batasan.

F. Rekaman Pengujian Per Konfigurasi

Tabel F.1 dan F.2 menampilkan rekaman mentah yang disalin dari berkas CSV hasil Colab, bukan pengukuran baru. Seluruh angka presisi penuh tersedia pada berkas aslinya di repositori GitHub.

Tabel F.1. Uji sensitivitas parameter, 45 percobaan pada data validasi.

H	LR	Seed	Akurasi (%)	Loss validasi
4	0.1	5	81.39	0.51641
4	0.1	11	91.61	0.26781
4	0.1	23	93.80	0.25532
4	0.1	37	95.99	0.20587
4	0.1	53	93.43	0.31337
4	0.5	5	97.81	0.06354
4	0.5	11	97.81	0.05232
4	0.5	23	97.81	0.05354
4	0.5	37	97.81	0.05142
4	0.5	53	97.81	0.05483
4	1.0	5	98.91	0.03801
4	1.0	11	98.91	0.03554
4	1.0	23	98.91	0.03640
4	1.0	37	98.91	0.03545
4	1.0	53	98.91	0.03727
8	0.1	5	93.43	0.24311
8	0.1	11	94.16	0.24223
8	0.1	23	91.24	0.25782
8	0.1	37	93.07	0.23179
8	0.1	53	93.07	0.27882
8	0.5	5	97.81	0.05025
8	0.5	11	97.81	0.04981
8	0.5	23	97.81	0.04978
8	0.5	37	97.81	0.04913
8	0.5	53	97.81	0.05281
8	1.0	5	98.91	0.03436
8	1.0	11	98.91	0.03497
8	1.0	23	98.91	0.03493
8	1.0	37	98.91	0.03479
8	1.0	53	98.91	0.03564
16	0.1	5	95.99	0.17776
16	0.1	11	93.80	0.20605
16	0.1	23	95.99	0.16701
16	0.1	37	94.89	0.18308
16	0.1	53	95.99	0.17161
16	0.5	5	97.81	0.04700
16	0.5	11	97.81	0.04800
16	0.5	23	97.81	0.04623
16	0.5	37	97.81	0.04806
16	0.5	53	97.81	0.04674
16	1.0	5	98.91	0.03326
16	1.0	11	98.91	0.03365
16	1.0	23	98.91	0.03240
16	1.0	37	98.91	0.03413
16	1.0	53	98.91	0.03266

Tabel F.2. Rekaman pengukuran waktu asli, 25 pengulangan.

N	Ulangan	Epoch	H	Waktu (ms/epoch)
164	1	120	8	0.149788
164	2	120	8	0.148021
164	3	120	8	0.148105
164	4	120	8	0.148587
164	5	120	8	0.163055
329	1	120	8	0.201759
329	2	120	8	0.123056
329	3	120	8	0.114391
329	4	120	8	0.115259
329	5	120	8	0.116065
493	1	120	8	0.146943
493	2	120	8	0.189949
493	3	120	8	0.140219
493	4	120	8	0.140652
493	5	120	8	0.141481
658	1	120	8	0.171001
658	2	120	8	0.169161
658	3	120	8	0.167927
658	4	120	8	0.168979
658	5	120	8	0.169548
823	1	120	8	0.196743
823	2	120	8	0.211670
823	3	120	8	0.196388
823	4	120	8	0.197232
823	5	120	8	0.195712

G. Bukti Perbandingan Turunan Analitik dan Numerik

Untuk setiap parameter θ dalam contoh 2–2–1, nilai gradien dari aturan rantai dapat dicocokkan dengan beda tengah $[L(\theta+\varepsilon) - L(\theta-\varepsilon)]/(2\varepsilon)$, menggunakan $\varepsilon = 10^{-5}$. Tabel G.1 adalah keluaran pemeriksaan numerik mandiri atas angka contoh. Mahasiswa perlu menghitung ulang dan menandai pemeriksaannya secara pribadi.

Tabel G.1. Nilai gradien berdasarkan Chain Rule dan beda tengah pada contoh manual.

Parameter	Analitik	Beda tengah	Selisih mutlak
W1(0, 0)	-0.056755138	-0.056755138	4.265e-12
W1(0, 1)	0.035304612	0.035304612	9.497e-13
W1(1, 0)	-0.113510275	-0.113510275	2.979e-12
W1(1, 1)	0.070609223	0.070609223	1.899e-12
b1(0, 0)	-0.056755138	-0.056755138	4.265e-12
b1(0, 1)	0.035304612	0.035304612	4.601e-12
W2(0, 0)	-0.320339481	-0.320339481	2.814e-12
W2(1, 0)	-0.381300126	-0.381300126	3.554e-12
b2(0, 0)	-0.496145516	-0.496145516	2.693e-12

Pada contoh ini selisih mutlak maksimum adalah 4.601e-12, lebih kecil dari ambang 10^{-8}. Tabel ini membuktikan konsistensi dua cara perhitungan pada titik yang diuji, bukan jaminan konvergensi Gradient Descent ataupun pembuktian akurasi Banknote.

Perhitungan memakai nilai parameter yang sama seperti Lampiran C. Kode pemeriksa yang dapat dijalankan berada di verifikasi_manual/periksa_gradien_manual.py dan output eksekusinya disimpan dalam hasil_pemeriksaan_numerik.txt.